An Enhanced Tendermint Consensus Protocol Powered by Elliptic Curve VRF for Beacon Chain Model

Ngoc Phuong Bui
phuong.bn212250M@sis.hust.edu.vn
Hanoi University of Science and Technology
Hanoi, Vietnam

Minh-Tri Hoang
trihm@tovchain.vn
Tovchain JSC
Hanoi, Vietnam

Thang Nguyen
thang@tovchain.vn
Tovchain JSC
Hanoi, Vietnam

Huu-Hai-Quan Dinh
quan@tovchain.vn
Tovchain JSC
Hanoi, Vietnam

Binh Minh Nguyen*
minhnb@soict.hust.edu.vn
Hanoi University of Science and Technology
Hanoi, Vietnam

ABSTRACT

Blockchain technology has seen rapid adoption of Proof-of-Stake consensus mechanism in lieu of Proof-of-Work due to the former's efficiency and speed. One notable example of Proof-of-Stake is the Tendermint protocol, which has been powering the entirety of Cosmos system - an ecosystem of multiple interlocked chains. However, Tendermint's choice of deterministically deciding the next block proposer presents a huge window for malicious actors to prepare and coordinate attacks on upcoming validator nodes, possibly crippling the attacked chain. Furthermore, randomized number generation in this blockchain ecosystem still proves to be a challenge by reason of blockchain's inherent deterministic nature. Aiming at the above problems, in this paper, we propose an improvement over Tendermint consensus protocol, utilizing Elliptic Curve Verifiable Random Function, a fast and secure pseudorandom generation algorithm suitable for deterministic systems like blockchain. This novel approach will solve the problem of knowing the validator ahead of time, whilst the verifiable random function module will be capable of supplying reliable random numbers to the overlaying beacon chain - a blockchain capable of distributing random numbers to users through smart contracts, and even other blockchains through Cosmos' InterBlockchain Communication Protocol. The performed experiments with a prototype blockchain demonstrated that the new consensus protocol improves Tendermint's resilience against network-layer attack vectors, while maintaining adequate fairness and performance.

CCS CONCEPTS

• **Computing methodologies** → *Distributed computing methodologies*;

*Corresponding author

SOICT 2023, December 07–08, 2023, Ho Chi Minh, Vietnam

ACM ISBN 979-8-4007-0891-6/23/12…$15.00
https://doi.org/10.1145/3628797.3628861

KEYWORDS

blockchain, consensus protocol, elliptic curve, verifiable random function, randomness

ACM Reference Format:
Ngoc Phuong Bui, Minh-Tri Hoang, Thang Nguyen, Huu-Hai-Quan Dinh, and Binh Minh Nguyen. 2023. An Enhanced Tendermint Consensus Protocol Powered by Elliptic Curve VRF for Beacon Chain Model. In *The 12th International Symposium on Information and Communication Technology (SOICT 2023), December 07–08, 2023, Ho Chi Minh, Vietnam.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3628797.3628861

1 INTRODUCTION

In 2008, a paper called "Bitcoin: A Peer-to-Peer Electronic Cash" was published by Satoshi Nakamoto, which proposed a novel type of decentralized system called blockchain [25]. The underlying infrastructure of Bitcoin's blockchain relies on Proof-of-Work (PoW) consensus protocol, where block builders (termed miners by the blockchain) compete for the construction of the next block and the corresponding reward, and PoW has been the go-to consensus mechanism for alternative chains such as Ethereum.

Recently, Proof-of-Stake (PoS) consensus protocol and its variants have been steadily being chosen instead of Proof-of-Work for blockchains, thanks to their efficiency at generating blocks and their speed of reaching consensus, allowing for greater versatility for blockchain networks while still maintaining fairness in reward distribution between block builders [22] [24]. One of the prominent variants of PoS is called Tendermint, which is a Byzantine Fault Tolerant (BFT)-style PoS mechanism [18]. Tendermint is widely adopted in the Cosmos ecosystem, which in turn is a network of interlocked chains that could communicate with each other using Cosmos' Inter Blockchain Communication (IBC) protocol [19] .

Yet, a possible problem of Tendermint lies in its choice of selecting future validator nodes for block proposals: an algorithm to deterministically pick the validator node to propose a block for a chosen block height based on the number of validators and the stake each validator has staked. As a result, malicious actors have full knowledge about the sequence of block proposers ahead of time, and could possibly orchestrate distributed attacks (e.g. Distributed Denial of Service (DDoS) or Eclipse Attack) against nodes right before they are selected to propose the block for the current height. Repeated attacks on various nodes will severely delay the consensus time and possibly cripple the entire chain as a result, of which small chains with low node counts are especially more vulnerable.

On the other hand, although PoS in general and Tendermint in particular could solve some of PoW problems such as energy wastefulness, due to blockchain's inherent deterministic nature, the answer for the outstanding issue of reliably generating random numbers within the blockchain remains elusive [11], [4]. Chainlink's attempt to solve this problem, that is utilizing an outside network to supply chains with Verifiable Random Number (VRF) [9], theoretically could work on any chain, but this leaves smart contracts and distributed applications heavily relying on a third party service. Meanwhile, the demand for random numbers on blockchain is quite big. Chainlink's VRF Coordinator on Binance Smart Chain - the smart contract that governs the dispense of random numbers - records as many as 5000 requests per day [2]. Thus, Cosmos Ecosystem would greatly benefit from having a source of randomness that is reliable and fast within its vast network - in other words, a random beacon chain - a blockchain that is capable of distributing verifiable random numbers through smart contracts, or to other blockchains via the InterBlockchain Communication Protocol.

In this paper, we propose an enhanced form of Tendermint Consensus Protocol to attempt to address the above issues. This novel approach utilizes Elliptic Curve Verifiable Random Function (ECVRF) to determine the next validator on every block creation, thus eliminating the danger of predetermined block proposer sequence as described above. Furthermore, the random module preinstalled inside the chain will act as a bridge to transport VRF results from the consensus layer to the smart contracts built on top of the blockchain and supply random numbers across the Cosmos Ecosystem through IBC protocol, effectively making the model a fast and reliable random number beacon chain.

The remainder of this paper is structured as follows. In Section 2, we will provide information about background knowledge related to the proposed model, namely Proof-of-Stake, Tendermint, Verifiable Random Function, Elliptic Curve, Eclipse Attack, and DDos Attack. The proposed model design will be presented in Section 3, and the implementation and evaluation of the design will be demonstrated in Section 4. Section 5 will conclude our research and outline future works.

2 RELATED WORKS

2.1 Proof-of-Stake and Tendermint Consensus Protocols

Proof-of-Stake protocol, where block creator is chosen based on the amount of "stake" they have put into the consensus mechanism, has been slowly but steadily replacing PoW, where nodes compete with each other for being chosen as the block creator through "work", as the choice of consensus for building blockchain, due to the former's superior speed and energy efficiency compared to the latter.

Although there exist many different methods of choosing the next block proposer for PoS, such as the Follow-the-Satoshi (FTS) method utilized by Tezos[5] or Ouroboros[17], or a weighted round robin algorithm utilized by Tendermint [23], a proposer selection method should satisfy two essential properties [6], that are:

(1) *Determinism*, which means at given validator set V, block height h and round r, the function $select(V, h, r)$ will return the same value, and provides that blockchain state is replicated.
(2) *Fairness*, which means given a validator set V with total voting power of P, a validator v with a voting power of p should be elected with a frequency of $\frac{p}{P}$ over an arbitrary long time.

In 2014, Kwon proposed a novel protocol called Tendermint that implements the Byzantine Fault Tolerant Proof-of-Stake scheme for confirming blocks [18].This protocol attempts to solve the "nothing at stake" problem of practical byzantine fault tolerance (PBFT) protocols, by making validators actually need to stake some of their assets to gain the right to vote for a new block, and they will be penalized if caught performing malicious acts.

The operation flow of Tendermint runs as follows: There are three steps in each round, namely Propose, Prevote, and Precommit. In Propose step, a deterministic round-robin algorithm is used to determine the proposer of the next block, and after the new block has been proposed, the validators will vote for that block in the Prevote step. If more than 2/3 of the voting power voted for a particular block, the consensus will enter Precommit step and will attempt to commit the validated block to the blockchain, thereby confirming all of the transactions included in that chain. If the system fails to reach a consensus (.e.g. less than 2/3 of the voting powers voted for that block, or more than 2/3 of the voting power broadcasted a nil vote), then a new round will begin until a new block is formed.

Tendermint is proven to be secure under the assumption that the network is partially synchronized as long as at least 2/3 of the voting power is honest. The protocol enjoys relatively high transaction throughput and low transaction confirmation time, therefore, it is currently the backbone of Cosmos Ecosystem, a network of blockchains built from Cosmos-SDK that are able to communicate with each other through the IBC protocol. Although the protocol is proven to be secure against cryptographic attacks, the deterministic nature of the round-robin proposer selection algorithm creates a window of opportunity for attackers to cripple the operation of the blockchain.

2.2 Elliptic Curve Verifiable Random Functions

Generating unpredictable randomized numbers on blockchain remains a daunting task. A candidate solution for solving this dilemma is called Verifiable Random Function (VRF), which was proposed by Micali et al. [21], wherein an entity could generate a pseudo-random number by utilizing a secret key S_k, and in turn, can prove the validity and correctness of such number by revealing the proof π and its public key P_k. In more detail, using the secret key S_k, a participant could compute the function $vrf(S_k, x)$ with the secret key and some input x of their choice, and, in turn, generate a pseudo-random number y with a proof π. Then, a skeptical user may confirm the correctness of y by computing the function $verify(P_k, \pi, x, y)$, which will return true if the computation is indeed correct. A secure VRF model should satisfy four distinct properties [15], that are:

(1) *Pseudorandomness*, which means the output y should appear to be statistically random, despite being the result of a deterministic function.
(2) *Verifiability*, which means the proof π will always be able to return true if the computation is correct.
(3) *Uniqueness*, which means for each P_k and input x, there cannot be different proofs π_1 and π_2 for different results y_1 and y_2.
(4) *Collision Resistance*, which means there cannot exist two distinct inputs x_1 and x_2 that result in the same output y.

An efficient construction of VRF is called Elliptic Curve VRF, where the secret-public key pairs are derived from elliptic curves [13]. This version of VRF has been proven to be fast, secure, and satisfies all of the criteria denoted above.

Chainlink VRF is a random number generation service developed by Chainlink [9]. This service enables users on blockchain to acquire safe and secure random numbers on blockchains, namely Ethereum and Binance Smart Chain. The user will first submit a request for a random number and a seed onto the blockchain through Chainlink's smart contract. Then, Chainlink will gather the requests, aggregate them, and deliver them to Chainlink's oracle, which is an off-chain external network of nodes, this oracle will generate the random numbers from the seeds provided along with the requests, verify the results, and submits the results back onto the blockchain through Chainlink's coordinator contract, where users can use the results on their own applications.

This approach indeed could provide provable-fair and unpredictable random numbers, but it relies heavily on Chainlink, and in case of service disruption (e.g. Chainlink decides to temporarily suspend operation), users will be unable to carry on normal operations. Furthermore, this approach limits users on Chainlink's supported networks, which are mainly EVM-based blockchains, not to mention the high cost associated with the requesting and receiving of random numbers from Chainlink's service.

2.3 Elliptic Curve Digital Signature

In blockchain systems, Elliptic Curve Digital Signature Algorithm (ECDSA) has been used predominantly for different cryptographic procedures, such as verifying transactions and creating digital signatures. ECDSA is derived from Digital Signature Algorithm (DSA) using a Koblitz elliptic curve, which is secp256k1 in Ethereum and Bitcoin blockchains [14]. In 2007, Edwards published a new normal form for elliptic curves [16], then Bernstein et al. generalized this Edwards form to twisted Edwards curves [8] with the equation:

$$x^2 + y^2 = 1 + dx^2y^2 \tag{1}$$

where $d \in k\backslash(0, 1)$. The sum of two points (x_1, y_1) and (x_2, y_2) on a twisted Edwards curve is

$$(x_1, y_1) + (x_2, y_2) = (\frac{x_1y_2 + y_1x_2}{1 + dx_1y_2y_1x_2}, \frac{y_1y_2 - x_1x_2}{1 - dx_1y_2y_1x_2}) \tag{2}$$

where the point (0,1) is the neutral element for addition, and an inversion of (x_1, y_1) returns $(-x_1, y_1)$.

In 2009, Bernstein proposed Curve25519 [7], which Bernstein also studied and confirmed its security and efficiency. This is the basis of Ed25519, a twisted Edwards form representation of Curve25519, which is used to construct the digital signature Ed25519.

Meryem et al. analyzed and evaluated Ed25519 against secp256k1, and confirmed that Ed25519 performs equally or better in at least two different methods of attack on the cryptographic level, and statistically performs faster than secp256k1 in both key pair generation and verification [12].

2.4 Eclipse and DDoS Attacks

There are numerous methods of attacks on blockchains, which could be broadly categorized into four groups: *Consensus-based Attacks*, *Peer-to-Peer Network-based Attacks*, *Smart Contract-based Attacks*, and *Wallet-based Attacks* [3]. We will focus on two methods of Peer-to-Peer Network-based Attacks that could potentially affect unsuspecting Proof-of-Work blockchains which we will discuss further below.

Eclipse Attack: Eclipse attack is a network-based attack that aims to isolate nodes from the entirety of the decentralized network. This is enacted by overrunning the routing table of the targeted node before forcing a restart, which will result in the victim node being forced to establish a connection with the attacker with high probability. From here, the attacker will have full control over the victim node's information flow and could perform malicious activities over the network. Due to the distributed nature of the blockchain, this attack method is focused on single nodes instead of the entire blockchain network at once.

DDoS Attack: DDoS Attack, short for Distributed Denial of Service Attack, is an attack where an attacker floods nodes with malicious requests in order to overwhelm the node, paralyzing it in the process. This attack is carried out by a widely distributed network of individual devices that is referred to as a botnet. At the designated time, the attacker will send a signal and each bot in the botnet will flood the victim node with multiple requests, overloading the node and crippling its ability to handle and process information from legitimate nodes, essentially excluding that node from the blockchain network.

Both attacks are capable of severely disrupting the work of the blockchain with coordination. For example, in January 2022, Solana suffered from a massive DDoS which caused widespread degradation throughout the network, forcing a rollback of the entire blockchain [10]. Thus, preventing these possible attack vectors remains an important task in designing and building new blockchains.

The works presented in this paper bring the following contributions:

(1) Improving Tendermint's consensus protocol, by adding a VRF generation and verification protocol on block creation for choosing the next proposer while maintaining adequate performance and ensuring the speed and efficiency of Tendermint consensus protocol remain.
(2) Proposing a random beacon chain design based on the random number generated by the VRF protocol, to be used by smart contracts within the VRF-enabled blockchain and other contracts within the Cosmos Ecosystem via IBC.
(3) Improving Chainlink's VRF by using a more secure elliptic curve.

3 DESIGNING THE CONSENSUS PROTOCOL

3.1 Proposal and PreVote Processes

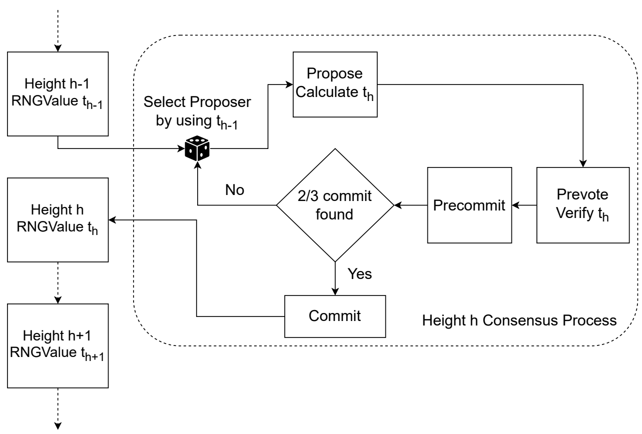

Figure 1: Consensus process at a given block height h

Based on the above discussions, we propose some improvements in design toward block proposer selection procedure, which can help mitigate future attacks, especially on small-scale blockchains with few validator nodes. The design is described in Figure 1

In our protocol, the validators also participate in the consensus process by signing votes for blocks proposed by the proposer, much like Tendermint protocol presented in subsection 2.1. Upon a new round of block formation at block height h, a new proposer for the next block is chosen. Instead of utilizing a deterministic function to select the proposer, the proposer is instead chosen by the following function:

$$proposer = getProposer(v, t_{h-1}, r) \tag{3}$$

in which v is the current validator set, t_{h-1} is the random number generated and included in the previous block, and r is the current round of the height h.

Since the function $getProposer(v, t_{h-1}, r)$ is deterministic and only cares about the values v, t_{h-1} and r which is publicly known, nodes can verify if they are selected independently for that round.

The proposer then forms the block, and also needs to generate a random value t_h. This is performed by first creating the alpha input:

$$m_r = keccak256(r, h-1, t_{h-1}) \tag{4}$$

in which $h-1$ is the height of previous confirmed block (0 if it is the first block), and t_{h-1} is the random value of the previous confirmed block (0 if it is the first block). In this, $keccak256$ is the hash algorithm keccak256, a secure hash algorithm that is based on sponge construction and demonstrates remarkable resistance against different attacks [20].

After that, the proposer combines m_r with its secret key S_k to generate the proof as follows:

$$\pi = ECVRF\_prove(S_k, m_r) \tag{5}$$

.

Algorithm 1 PROPOSAL

Input: height h, transactions txs_h, previous height $h-1$, validator set v and previous random number t_{h-1}
Output: proposal $propose(h, r, block_h, \pi_h, t_h)$ or $timeout(round_r)$.

```
When it is time for NewHeight:
Initialize round r with height h, transactions txs_h, previous
  height h − 1, validator set v and previous random number t_{h−1}
Calculate proposer p_(h,r) by using getProposer(v, t_{h−1}, r)
if p_(h,r) = p then
    block_h = createBlock(txs_h)
    m_r = computeInput(r, h − 1, t_{h−1})
    π_h = ECVRF_prove(S_k, m_r)
    t_h = ECVRF_proof_to_hash(π_h)
    Broadcast propose(h, r, block_h, π_h, t_h) to adjacent nodes
else if p_(h,r) ≠ p then
    Prepare for timeout(round_r)
end if
```

Finally, the proposer confirms the final random value by the following formula:

$$t_h = ECVRF\_proof\_to\_hash(\pi) \tag{6}$$

and attaches its public key P_k and the result t_h to the proposal, which are broadcasted over the blockchain via the Gossip protocol to adjacent nodes. When each validator verifies that over two-thirds of the nodes have acknowledged the proposal or a timeout limit has been reached, the validator enters the prevote step.

In the prevote step, the validator verifies two components. First, it verifies that the proposed block is a valid one, by running an external function $isBlockValid = validate(block_h)$. If $isBlockValid = true$, then the validator verifies the random number generated by the proposer by first computing the alpha input using function (4) then the validator runs the following verification function:

$$(isVRFValid, t'_h) = ECVRF\_verify(P_k, \pi, m_r) \tag{7}$$

in which $t'_h = ECVRF\_proof\_to\_hash(\pi)$.

Since each combination of S_k and m_r generates a unique proof π, function (7) only returns **isVRFValid** = **true** if the proof π (and, in turn, the value t_h) is calculated in an honest manner by the proposal. In the last stage, if both $isBlockValid$ and $isVRFValid$ return $true$ and the value t'_h is equal to t_h, the validator will sign, submit a prevote for the proposed block and broadcast this prevote to the adjacent validators through Gossip protocol. In any other case (e.g. $isVRFValid$ returns false), the validator will sign a *nil* message and broadcast this prevote to the blockchain. The rest of the protocol proceeds according to the original Tendermint's protocol.

At the end of each height, the block committed to the blockchain at height h will have the random value t_h and proof π, whose correctness is verified by participating validators, inscribed onto it, and upon each of the rounds in the consensus process of the next block, this value will be used to determine the next proposer.

This process ensures that a malicious actor cannot predetermine block proposer of height h for an indefinite amount of time ahead of height h, but can only know the possible proposer of height h ahead by at most height $h-1$, preventing him from planning,

Algorithm 2 PREVOTE

Input: proposal $propose(h, r, block_h, \pi_h, t_h)$
Output: prevote $prevote(h_p, r, id(block_h))$

When validator receives $propose(h, r, block_h, \pi_h, t_h)$:
Calculate P_k by using $getPK(p_{(h,r)})$
Calculate $isBlockValid$ by using $validate(block_h)$
Calculate m_r by using $computeInput(r, h-1, t_{h-1})$
Calculate $(isVRFValid, t'_h)$ by using $ECVRF\_verify(P_k, \pi_h, m_r)$
if $isBlockValid \wedge isVRFValid \wedge t'_h = t_h$ **then**
    Broadcast $prevote(h_p, r, id(block_h))$ to adjacent nodes ▷ $id(block)$ is the unique hash of $block$
else
    Broadcast $prevote(h_p, r, nil)$ to adjacent nodes
end if

coordinating and scheduling attacks against upcoming proposers and disrupting the network, ensuring the integrity of the system.

3.2 Random Module and Beacon Chain

Another improvement that we have devised for this random beacon chain model is the inclusion of the random module, which retrieves the VRF numbers of the consensus layer and broadcasts it to the application layer, allowing users to also utilize this random result. This design is described in Figure 2

This module has the IBC capability of Cosmos, and is fully compatible with the CosmWasm module, as a result, this module is capable of supplying random numbers to smart contracts deployed on other blockchains in the ecosystem and enables this blockchain to act as a random beacon chain inside the Cosmos Ecosystem, for example, for a lottery game that is deployed on the blockchain.

One example of how this module can be used is when a user, for example, Alice, wants to deploy a lottery system. She first writes a smart contract, then specifies which block height will be used for the random number retrieval (which, of course, should be a future block, otherwise it will defeat the purpose of random number). Then, at the time of the lottery draw, the smart contract retrieves the random number t_h from the predetermined block and calculates the outcome. Anyone could verify the correctness of t_h by simply calculating input m_r and using it along with P_k and π_h.

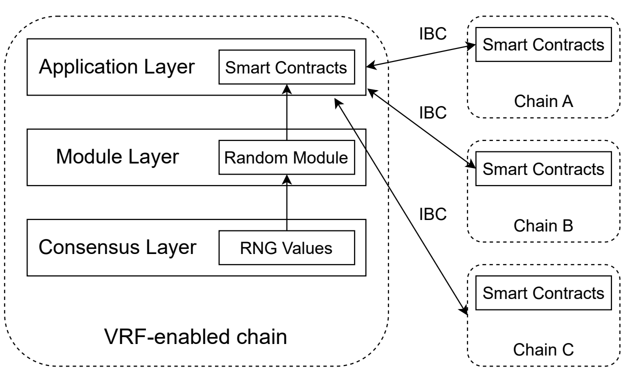

Figure 2: Random module design

One could argue that the validator can possibly manipulate the result of a random number produced at height h to manipulate the VRF number for smart contracts, but the $ECVRF\_verify(P_k, \pi_h, m_r)$ function's inputs are all determined and publicly known, which means the outcome t_h is impossible to manipulate unless the proposer intentionally forfeits his rights to propose that block, potentially facing a stake slash and still cannot ensure that the outcome will be favorable to him.

This approach towards random number generation on blockchain is different from Chainlink's VRF method in two ways. Firstly, the proposed random beacon chain model's random number generation is a by-product of the consensus itself, whose validators have already been rewarded by the consensus algorithm, thus users can use these random numbers without charge unlike Chainlink, and as long as all of the nodes inside the random beacon chain is working properly, users will not have to worry about service disruption. Secondly, the random beacon chain's ECVRF construction uses the Ed25519 elliptic curve instead of Chainlink's secp256k1, which has been proven to house numerous advantages, both in security and efficiency as described in Section 2.

4 IMPLEMENTATION AND EVALUATION

To implement the proposed method as described in section 3, we constructed a prototype blockchain and will assess it for the qualities of both its selection method correctness and block formation time, then compare it to Tendermint's.

4.1 Environmental Setup

All of the experiments are conducted on four Docker containers deployed on a local network with four distinct IP addresses and ports to avoid network collision. All are run on the same hardware configuration: a CPU of 2GHz Quad-Core Intel Core i5 and 16GB of RAM.

The DDoS attacks are performed using Apache Benchmark [1], which simultaneously generates 60.000 concurrent requests each interval to the targeted node IP address and port. The number is chosen considering the limits of the testing environment's bandwidth and computer resources.

The simulated attack is run on a blockchain comprises of 4 nodes, all of them have the same voting power to ensure the proposal chance is distributed equally. The attacks last for an arbitrary amount of block, then we take a sample of 1000 continuous blocks for measurement.

4.2 Correctness

To evaluate the proposed consensus protocol variant against a coordinated DDoS attack, we ran a simulated DDoS attack against a prototype comprised of 4 nodes and then compared it against Tendermint's similar setup. Since Tendermint's proposer selection algorithm is deterministic and in a round-robin fashion, the coordinated attack will know beforehand which node is being selected next and is programmed in a predetermined pattern. When determined a block height to start attacking, the attack algorithm will calculate the designated block height's proposer node and launch

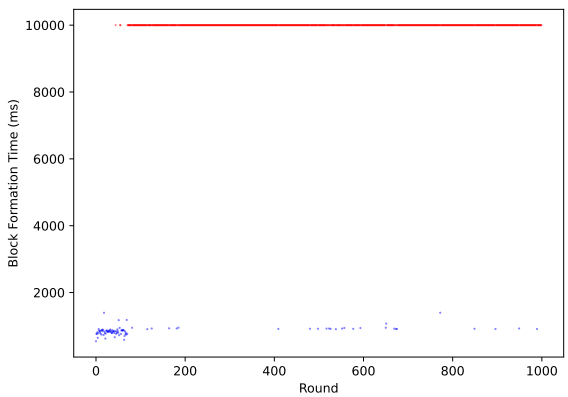

Figure 3: Average block formation time under coordinated DDoS attack of Tendermint consensus protocol

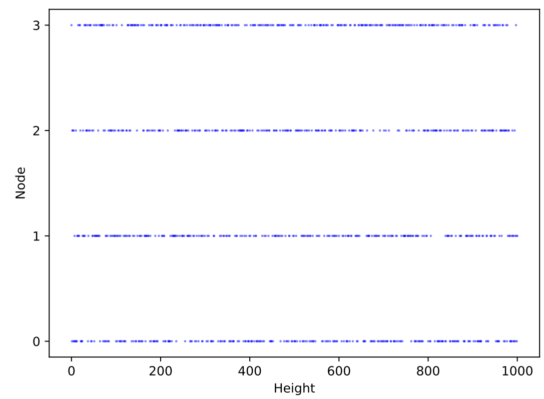

Figure 5: Random beacon chain's scatter plot of node selection over 1000 blocks for 4 nodes, each with 1 voting power

an attack on that node when the time comes. After that, the attack algorithm will attack subsequent proposers at each timeout interval according to the round-robin selection algorithm.

Because the proposed model's selection algorithm uses VRF, the pattern cannot be determined beforehand, thus we applied the same pattern as Tendermint's attack to compare the results. At a determined block height, the attack algorithm will pick the proposer according to Tendermint's selection algorithm, and then proceed with the attack accordingly.

We then gathered the average block formation time of the nodes and created a scatter plot. A block formation time of 10000ms means the protocol timed out waiting for the block proposer to initialize a block.

From the results, we can observe that under coordinated attack, the blockchain constantly reaches timeout and needs to advance to new rounds, significantly increasing block creation time. Meanwhile, due to the unpredictability of the new model's consensus algorithm, the coordinate attacks cannot always target the new model chain's proposal nodes, allowing the network to function properly even with one node losing its functionality. There are still occasions when the proposal node is the targeted node, and in such cases, the network will timeout and need to move to a new round, but compared to Tendermint's consensus protocol, the successful consensus rate is significantly higher.

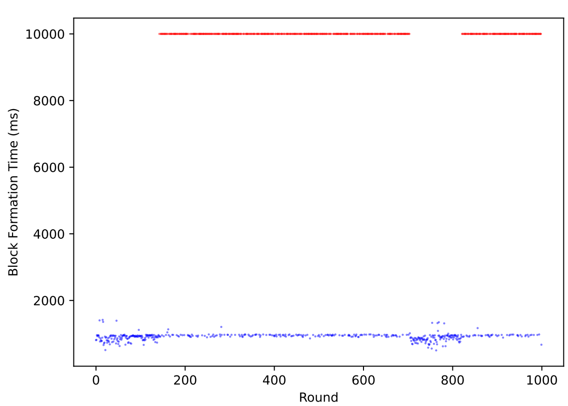

Figure 4: Average block formation time under coordinated DDoS attack of VRF-enabled consensus protocol

Out of 1000 rounds, 92.5% of Tendermint's consensus protocol rounds reached timeout without being able to finalize blocks when under coordinated attacks, meanwhile the figure is only 52.8% with the proposed variant, as seen by the distributions observed in Figure 3 and 4 - in which the red dots denote timed out rounds and the blue dots denote the average block formation time of completed block.

4.3 Fairness

As discussed in subsection 2.1, Proof-of-Stake selection process should satisfy two essential qualities, *Determinism* and *Fairness*.

(1) *Determinism*: The choice of implementing ECVRF using the elliptic curve already ensures the determinism of the consensus, since function (4) and sequentially function (5) are both deterministic.
(2) *Fairness*: One of the key values of ECVRF lies in its full pseudorandomness, which was proven by S. Goldberg et al [15]. In short, ECVRF algorithm ensures that someone without knowledge of secret key S_k will not be able to distinguish the outcome t_h from a random value. Furthermore, the proposer cannot choose the input m_r, but needs to conspicuously perform a predetermined function (4) to generate the VRF number.

We performed a pseudorandom distribution test, then we performed a test on the prototype with 4 nodes and analyzed the results of block proposer selection over a range of approximately 10,000 blocks multiple times to verify the fairness of the consensus.

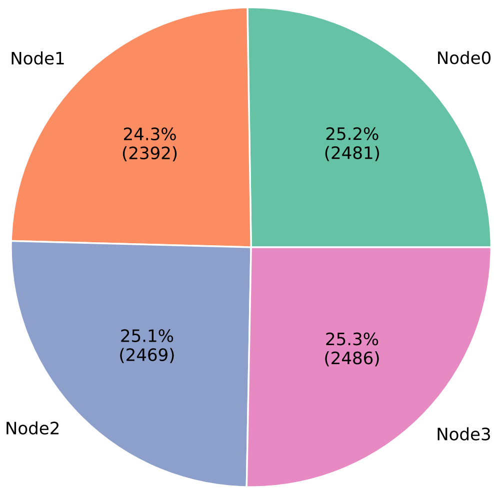

Figure 6: VRF-enabled consensus protocol's percentage of node selection over 9852 blocks for 4 nodes, each with 1 voting power

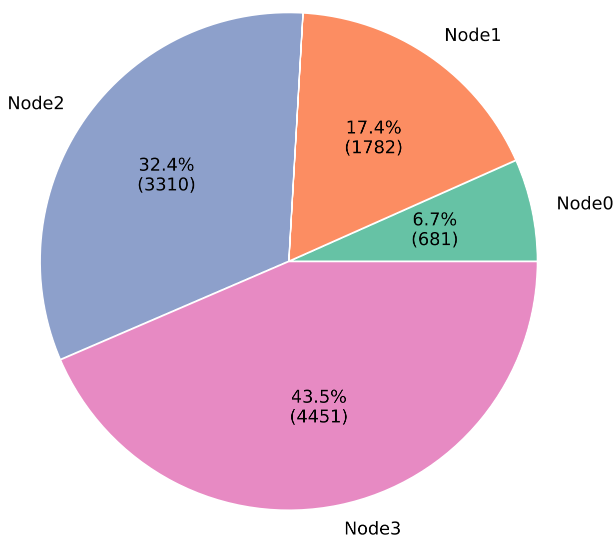

Figure 7: VRF-enabled consensus protocol's percentage of node selection over 10224 blocks for 4 nodes, with voting powers equal to 1, 3, 5, 7 respectively

From the scatter plot depicted in Figure 5, it is shown that the algorithm will distribute the selection throughout 4 nodes in a fairly even manner, without many gaps or clusters. Over a longer period of time, this distribution will gradually reach a discrete uniform distribution.

This is backed by the data shown in Figure 6 and 7. With a testing sample of approximately 10,000 blocks, the chance of a block being selected as the proposer is an approximation of the function:

$$c_i = \frac{p_i}{P} = \frac{p_i}{\sum_{i=0}^{n-1} p_i} \tag{8}$$

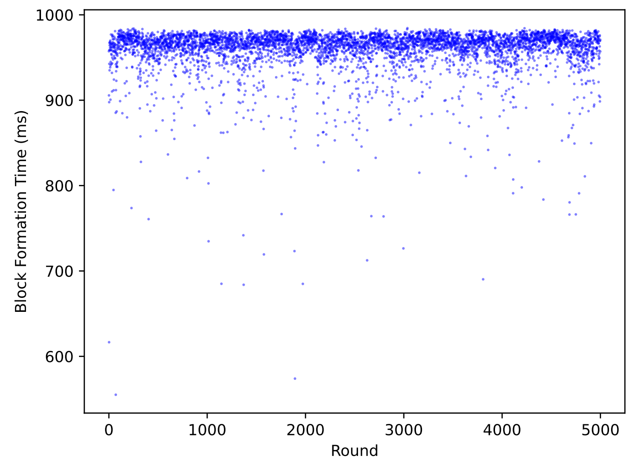

Figure 8: VRF-enabled consensus protocol's node0 block formation time measurements over length of 5000 blocks

in which p_i is the voting power of node i. In an arbitrarily long sequence of blocks, this approximation should gradually reach the ratio of the round-robin algorithm that Tendermint has developed.

For the above experiments, we see that this random beacon chain model's consensus algorithm satisfies both determinism and fairness for the proposer selection process, and adds an extra layer of protection against potential attack vectors.

4.4 Performance

To evaluate the performance of this random beacon chain model's consensus algorithm and the impact of VRF calculation, we will conduct several tests over the length of approximately 10000 blocks to measure the block finalization time of each node on a constant input stream of roughly 3 transactions per second.

Since validator nodes can perform the selection function independently, there exists no extra communication delay between nodes, and there is only a slight overhead of 12ms for each node due to the calculation and verification of VRF, as shown in Table 1.

We also plotted a graph of individual duration measurements of block formation time on two nodes over 5000 blocks. From Figure 8 and 9, we can see that the block finalization duration of node0 and node1 stay mostly stable throughout a sequence of 5000 blocks and a constant input rate of 3 transactions per second, with some occasional anomalies, but there recorded no timeout and no severely delayed block finalization time.

Table 1: Average block finalization duration of nodes over 10000 blocks (in ms)

Node	VRF-powered Chain	Tendermint
Node 0	958.333121780022	945.4388912947659
Node 1	958.1333696459159	946.0899973030309
Node 2	958.2167081699193	946.0843910854002
Node 3	958.4829165905359	946.0987231019282

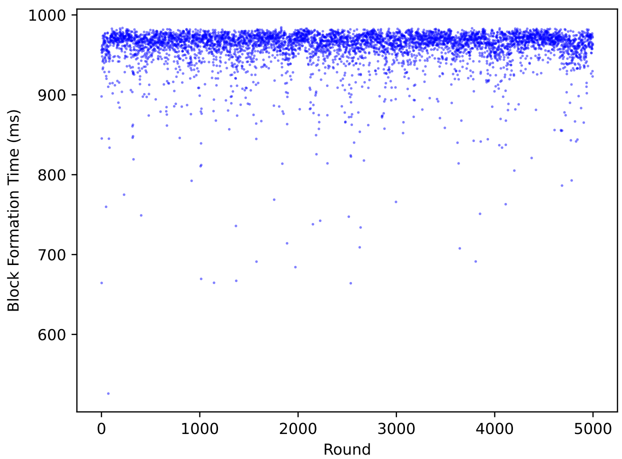

Figure 9: VRF-enabled consensus protocol's node1 block formation time measurements over length of 5000 blocks

From these results, we concluded that the addition of ECVRF into the consensus procedure does not cause critical degradation or severe delay in block formation, and the performance of the random beacon chain's consensus algorithm is comparable to that of Tendermint's while keeping the selection procedure safer and more secure.

5 CONCLUSION AND FUTURE WORKS

This paper proposed a novel random beacon chain model powered by a provably fair ECVRF-powered consensus, which, as demonstrated through experiments, possesses the capability to mitigate network-based attack vectors while retaining the determinism, fairness, and performance of the Tendermint consensus protocol. Furthermore, the preinstalled random module built underlying the Cosmwasm module allows this random beacon chain to act as a random beacon chain, both to serve provably fair random numbers to smart contracts deployed within the network and to serve random numbers for adjacent blockchains throughout the Cosmos Ecosystem through the IBC protocol.

Future works in this area involve developing on-demand number generation requests, i.e. allowing validators to process random number requests from smart contracts much like Chainlink's model, and aiming to further improve the performance of this novel random beacon chain model.

ACKNOWLEDGEMENTS

This research was supported by the Center for Digital Technology and Economy (BK Fintech) of Hanoi University of Science and Technology (HUST), Tovchain JSC, and Trava Finance project. We would like to present our sincere gratitude to these organizations and their staff for sharing valuable experiences with us during the course of the research.

REFERENCES

[1] 2023. Apache HTTP server benchmarking tool. https://httpd.apache.org/docs/2.4/programs/ab.html Last accessed: 2023-09-12.

[2] 2023. ChainlinkVRFCoordinator Smart Contract. https://bscscan.com/address/0xc587d9053cd1118f25F645F9E08BB98c9712A4EE Last accessed: 2023-09-12.

[3] Shubhani Aggarwal and Neeraj Kumar. 2021. Chapter Twenty - Attacks on blockchain - Working model. In *The Blockchain Technology for Secure and Smart Applications across Industry Verticals*, Shubhani Aggarwal, Neeraj Kumar, and Pethuru Raj (Eds.). Advances in Computers, Vol. 121. Elsevier, 399–410. https://doi.org/10.1016/bs.adcom.2020.08.020

[4] Peter Alleman. 2021. Randomness and Games on Ethereum. https://crypto.unibe.ch/archive/theses/2021.msc.peter.allemann.pdf

[5] Victor Allombert, Mathias Bourgoin, and Julien Tesson. 2019. Introduction to the Tezos Blockchain. 1–10. https://doi.org/10.1109/HPCS48598.2019.9188227

[6] Antonina Begicheva and A Kofman. 2018. Fair Proof of Stake. (05 2018). https://doi.org/10.13140/RG.2.2.11204.37765

[7] Daniel J. Bernstein. 2006. Curve25519: New Diffie-Hellman Speed Records. In *Public Key Cryptography - PKC 2006*, Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 207–228.

[8] Daniel J. Bernstein and Tanja Lange. 2007. Faster Addition and Doubling on Elliptic Curves. In *Advances in Cryptology – ASIACRYPT 2007*, Kaoru Kurosawa (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 29–50.

[9] Lorenz et al. Breidenbach. 2021. Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Network. https://naorib.ir/white-paper/chinlink-whitepaper.pdf

[10] Rajasekhar Chaganti, Rajendra V. Boppana, Vijayakumar Ravi, Kashif Munir, Mubarak Almutairi, Furqan Rustam, Ernesto Lee, and Imran Ashraf. 2022. A Comprehensive Review of Denial of Service Attacks in Blockchain Ecosystem and Open Challenges. *IEEE Access* 10 (2022), 96538–96555. https://doi.org/10.1109/ACCESS.2022.3205019

[11] Krishnendu Chatterjee, Amir Kafshdar Goharshady, and Arash Pourdamghani. 2019. Probabilistic Smart Contracts: Secure Randomness on the Blockchain. In *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. 403–412. https://doi.org/10.1109/BLOC.2019.8751326

[12] Meryem Cherkaoui Semmouni, Nitaj Abderrahmane, and Mostafa Belkasmi. 2019. *Bitcoin Security with a Twisted Edwards Curve*. Technical Report. https://normandie-univ.hal.science/hal-02320909v1/file/EdwardsBitcoinFinal-V3.pdf

[13] Yevgeniy Dodis and Aleksandr Yampolskiy. 2005. A Verifiable Random Function with Short Proofs and Keys. In *Public Key Cryptography - PKC 2005*, Serge Vaudenay (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 416–431.

[14] Johnson Don, Menezes Alfred, and Vanstone Scott. 2001. The Elliptic Curve Digital Signature Algorithm (ECDSA). In *International Journal of Information Security*. Springer.

[15] S. Golberg and L. Reyzin. 2023. *Verifiable Random Function*. Technical Report. https://www.rfc-editor.org/rfc/rfc9381.pdf

[16] M. Edwards Harold. 2007. A normal form for elliptic curves. In *Bulletin of the American Mathematical Society 44*. AMS, 393–422.

[17] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. 2017. Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol. In *Advances in Cryptology – CRYPTO 2017*, Jonathan Katz and Hovav Shacham (Eds.). Springer International Publishing, Cham, 357–388.

[18] Jae Kwon. 2014. *Tendermint: Consensus without Mining*. Technical Report. https://tendermint.com/static/docs/tendermint.pdf

[19] Jae Kwon and Ethan Buchman. 2019. Cosmos Whitepaper. https://wikibitimg.fx994.com/attach/2020/12/16623142020/WBE16623142020_55300.pdf

[20] Jiasong Liu. 2023. Digital signature and hash algorithms used in Bitcoin and Ethereum. In *Third International Conference on Machine Learning and Computer Application (ICMLCA 2022)*, Shuhong Ba and Fan Zhou (Eds.), Vol. 12636. International Society for Optics and Photonics, SPIE, 126365H. https://doi.org/10.1117/12.2675431

[21] S. Micali, M. Rabin, and S. Vadhan. 1999. Verifiable random functions. In *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*. 120–130. https://doi.org/10.1109/SFFCS.1999.814584

[22] P.R. Nair and D.R. Dorai. 2019. Evaluation of Performance and Security of Proof of Work and Proof of Stake using Blockchai. In *Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks*. IEEE, 279–283.

[23] Cong T. Nguyen, Dinh Thai Hoang, Diep N. Nguyen, Dusit Niyato, Huynh Tuong Nguyen, and Eryk Dutkiewicz. 2019. Proof-of-Stake Consensus Mechanisms for Future Blockchain Networks: Fundamentals, Applications and Opportunities. *IEEE Access* 7 (2019), 85727–85745. https://doi.org/10.1109/ACCESS.2019.2925010

[24] Moritz Platt, Johannes Sedlmeir, Daniel Platt, Jiahua Xu, Paolo Tasca, Nikhil Vadgama, and Juan Ignacio Ibañez. 2021. The Energy Footprint of Blockchain Consensus Mechanisms Beyond Proof-of-Work. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. 1135–1144. https://doi.org/10.1109/QRS-C55045.2021.00168

[25] Nakamoto Satoshi. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf